



University of Copenhagen
MSc. Physics

Continuous Time Finance 2

Hand-In 3

Author:

Benjamin Tariq Siddique

Contact: vhb925@alumni.ku.dk
ben123sid@gmail.com

1 – Portfolio insurance, static hedging and stochastic volatility

1.a

Using Björk's Lemma 6.12 [1], we may write dA_t^a as

$$dA_t^a = A_t^a \cdot \left[w_t^S \frac{dS_t^i}{S_t^i} + w_t^B \frac{dB_t^i}{B_t^i} \right].$$

In our case clearly $w_t^S = a$ and $w_t^B = (1 - a)$. We may thus write the above formula as

$$\begin{aligned} dA_t^a &= A_t^a \cdot [a(\mu dt + \sigma dW_t^P) + (1 - a)rdt], \\ &= A_t^a \cdot [(a\mu + r - ar)dt + a\sigma dW_t^P]. \end{aligned}$$

Thus under the P -measure we have drift $\mu^P = a\mu + r - ar$ and volatility $\sigma^P = a\sigma$. Under the risk-neutral Q -measure we instead have

$$\begin{aligned} dA_t^a &= A_t^a \cdot [a(rdt + \sigma dW_t^Q) + (1 - a)rdt], \\ &= A_t^a \cdot [(ar + r - ar)dt + a\sigma dW_t^Q], \\ &= A_t^a \cdot [rdt + a\sigma dW_t^Q]. \end{aligned}$$

And thus the drift and volatility are $\mu^Q = r$ and $\sigma^Q = \sigma$ respectively. Now consider the fact that we know that A_t^a is a geometric brownian motion process, that we may write as

$$A_t^a = A_0 \exp \left[\left(a\mu + r - ar - \frac{1}{2}a^2\sigma^2 \right) t + a\sigma dW_t^P \right].$$

In the same way we may write

$$(S_t)^a = (S_0)^a \exp \left[\left(a\mu - \frac{1}{2}a^2\sigma^2 \right) t + a\sigma dW_t^P \right].$$

Now it should be obvious that we may write $A_t^a = g(t)(S_t)^a$, where

$$g(t) = \frac{A_0}{(S_0)^a} \exp \left[\left(r - ar + \frac{1}{2}a\sigma^2 - \frac{1}{2}a^2\sigma^2 \right) t \right].$$

1.b

Consider that A_t^a has the Q -dynamics

$$dA_t^a = rA_t^a dt + a\sigma A_t^a dW_t^Q.$$

Thus it should be trivial to see that $(K - A_T^a)^+$ is simply a european put option, which its price given by the Black-Scholes Formula

$$\Pi(t, A) = e^{-r(T-t)} KN[-d_2(t, A)] - AN[-d_1(t, A)],$$

where $N[\cdot]$ is the cumulative normal distribution function and

$$d_1(t, A) = \frac{1}{\sigma\sqrt{T-t}} \left\{ \ln \left(\frac{A}{K} \right) + \left(r + \frac{1}{2}a^2\sigma^2 \right) (T-t) \right\},$$

$$d_2(t, A) = d_1(t, A) - a\sigma\sqrt{T-t}.$$

Thus for the given parameters $S_0 = A_0^a = 1$, $T = 30$, $K = e^{rT}$ and $a = 0.5$ we get a price of

$$\Pi \approx 0.2158.$$

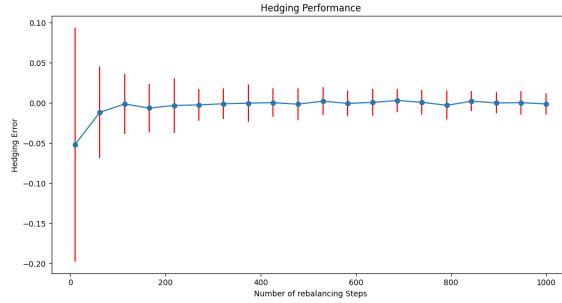
The contract can be replicated through a dynamic delta-hedging strategy. The delta for a BS put option is given in Björk exercise 10.10 as

$$\Delta_{\text{put}} = \frac{\partial \Pi}{\partial A^a} = N[d_1] - 1.$$

Note that we can however not directly trade A^a , since it is a portfolio and not a stock. Instead it would be more productive to consider

$$\Delta = \frac{\partial \Pi}{\partial S} = \frac{\partial \Pi}{\partial A^a} \frac{\partial A^a}{\partial S} = (N[d_1] - 1)a \frac{A_t^a}{S_t},$$

where we used the previous result $A_t^a = g(t)(S_t)^a$ for the differentiation. Thus we can replicate the contract by holding $(N[d_1] - 1)a \frac{A_t^a}{S_t}$ units of stock and investing the remainder in a risk-free asset. This is shown by reusing some of the code from Hand-In 1 (with minor tweaks) and plotting the hedging error against the number of rebalancing steps.



As seen in the figure, the hedging error converges to 0 as $n \rightarrow \infty$, showing that this delta-hedging-strategy successfully replicates the contract. The red bars denote the standard deviation of the hedging errors, as each point represents 100 simulated option paths.

1.c

- $e^{-rT} \mathbb{E}^P[(K - A_T^a)^+]$

Consider that the above can be expressed through Björk's eq. 7.51 with the following tweaks

$$F(0, A) = e^{-rT} \int_{-\infty}^{\ln(\frac{K}{A_0})} (K - A_0^a e^z) f(z) dz, \quad \text{where } f(z) \sim N\left(\left(a\mu + r - ar - \frac{1}{2}a^2\sigma^2\right)T, a^2\sigma^2T\right).$$

Handling the first term:

$$\int_{-\infty}^{\ln(\frac{K}{A_0})} K f(z) dz = \frac{K}{\sqrt{2\pi a^2 \sigma^2}} \int_{-\infty}^{\ln(\frac{K}{A_0})} e^{-\frac{(z - (a\mu + r - ar - \frac{1}{2}a^2\sigma^2)T)^2}{2a^2\sigma^2T}} dz$$

Substitute $y = \frac{z - (a\mu + r - ar - \frac{1}{2}a^2\sigma^2)T}{a\sigma\sqrt{T}}$ with $dy = \frac{1}{a\sigma\sqrt{T}} dz$:

$$\frac{K}{\sqrt{2\pi}} \int_{-\infty}^{\frac{\ln(\frac{K}{A_0}) - (a\mu + r - ar - \frac{1}{2}a^2\sigma^2)T}{a\sigma\sqrt{T}}} e^{-\frac{y^2}{2}} dy = \frac{K}{\sqrt{2\pi}} N\left[\frac{\ln\left(\frac{K}{A_0}\right) - (a\mu + r - ar - \frac{1}{2}a^2\sigma^2)T}{a\sigma\sqrt{T}}\right]$$

Now for the second term, we make the same substitution:

$$\int_{-\infty}^{\ln\left(\frac{K}{A_0}\right)} A_0^a e^z f(z) dz = \frac{A_0^a}{\sqrt{2\pi}} \int_{-\infty}^{\frac{\ln\left(\frac{K}{A_0}\right) - (a\mu + r - ar - \frac{1}{2}a^2\sigma^2)T}{a\sigma\sqrt{T}}} e^{-\frac{y^2}{2} + a\sigma\sqrt{T}y + (a\mu + r - ar - \frac{1}{2}a^2\sigma^2)T} dy$$

Completing the square simplifies the integral to

$$\frac{A_0^a}{\sqrt{2\pi}} \int_{-\infty}^{\frac{\ln\left(\frac{K}{A_0}\right) - (a\mu + r - ar - \frac{1}{2}a^2\sigma^2)T}{a\sigma\sqrt{T}}} e^{-\frac{(y - a\sigma\sqrt{T})^2}{2} + (a\mu + r - ar - \frac{1}{2}a^2\sigma^2)T + \frac{1}{2}a^2\sigma^2T} dy,$$

which can be further simplified using the substitution $x = y - a\sigma\sqrt{T}$ with $dy = dx$:

$$\frac{A_0^a}{\sqrt{2\pi}} e^{(a\mu + r - ar)T} \int_{-\infty}^{\frac{\ln\left(\frac{K}{A_0}\right) - (a\mu + r - ar - \frac{1}{2}a^2\sigma^2)T}{a\sigma\sqrt{T}} - a\sigma\sqrt{T}} e^{-\frac{x^2}{2}} dx.$$

Let $u_1 = \frac{\ln\left(\frac{K}{A_0}\right) - (a\mu + r - ar - \frac{1}{2}a^2\sigma^2)T}{a\sigma\sqrt{T}}$, then our final expression becomes:

$$\begin{aligned} e^{-rT} \mathbb{E}^P[(K - A_T^a)^+] &= e^{-rT} \left(KN[u_1] - A_0^a e^{(a\mu + r - ar)T} N\left[u_1 - a\sigma\sqrt{T}\right] \right), \\ &= e^{-rT} KN[u_1] - A_0^a e^{(a\mu - ar)T} N\left[u_1 - a\sigma\sqrt{T}\right]. \end{aligned}$$

Calculating this expression yields $e^{-rT} \mathbb{E}^P[(K - A_T^a)^+] \approx 0.0304$, which is way lower than the risk-neutral price obtained in the previous section. It makes sense that this value is lower, since the real-world drift of A_t^a is much higher and therefore this put is less likely to be exercised, given the risk profile.

- $e^{-rT} \mathbb{E}^Q[(K - (S_T)^a)^+]$

By retracing the steps above and noting that the process can be written under the Q-measure as

$$(S_t)^a = (S_0)^a \exp \left[\left(ar - \frac{1}{2}a\sigma^2 \right) t + a\sigma dW_t^Q \right],$$

it is easy to transfer the drift and volatility terms to find

$$\begin{aligned} e^{-rT} \mathbb{E}^P[(K - (S_T)^a)^+] &= e^{-rT} \left(KN[v_1] - A_0^a e^{(ar - \frac{1}{2}\sigma(a - a^2))T} N\left[v_1 - a\sigma\sqrt{T}\right] \right), \\ &= e^{-rT} KN[v_1] - A_0^a e^{(ar - r - \frac{1}{2}\sigma(a - a^2))T} N\left[v_1 - a\sigma\sqrt{T}\right], \end{aligned}$$

where $v_1 = \frac{\ln\left(\frac{K}{(S_0)^a}\right) - (ar - \frac{1}{2}\sigma^2 a)T}{a\sigma\sqrt{T}}$ we find that $e^{-rT} \mathbb{E}^Q[(K - (S_T)^a)^+] \approx 0.4119$, which is higher than the price found in 1b. This can be explained by the fact that its drift term ignores the cash earned from the risk-free asset (which can clearly be seen in the drift from dA_t^a).

- $ae^{-rT} \mathbb{E}^Q[(K - S_T)^+]$

This is clearly just a normal european put so we can just use the classical Black-Scholes formula to find

$$\begin{aligned} e^{-rT} \mathbb{E}^P[(K - A_T^a)^+] &= ae^{-rT} (KN[-d_2] - S_0 e^{rT} N[-d_1]) \\ &= a(K e^{-rT} N[-d_2] - S_0 N[-d_1]). \end{aligned}$$

It has a value of $e^{-rT} \mathbb{E}^P[(K - A_T^a)^+] \approx 0.2081$. This is slightly smaller than the price found in 1b) and highlights the fact that simply betting a fraction of your wealth a of the stock-price to fall below K , is not the same as betting against a portfolio A_t^a , whose stock-proportion is continuously rebalanced.

1.d

Let $f(S)$ be a twice differentiable function, such that we may Taylor expand it around S_0 :

$$f(S) = f(S_0) + f'(S_0)(S - S_0) + \int_{S_0}^S f''(K)(S - K)dK,$$

where the last term represents the remainder. The goal is now to transform that integral into a call- and a put-term, like in Carr & Madan eq. (1). Consider first the case when $S > S_0$: We may then substitute the $(S - K)$ -factor with the call-payoff $(S - K)^+$ as well as shift the limits of the integral from $[S_0, S]$ to $[S_0, \infty]$:

$$\int_{S_0}^S f''(K)(S - K)dK = \int_{S_0}^{\infty} f''(K)(S - K)^+ dK.$$

Note that this integral is 0 if $S < S_0$. Consider then in that case ($S < S_0$), that we may flip the limits, take the payoff function, and extend the lower limit to 0:

$$\int_{S_0}^S f''(K)(S - K)dK = - \int_S^{S_0} f''(K)(S - K)dK = \int_S^{S_0} f''(K)(K - S)dK = \int_0^{S_0} f''(K)(K - S)^+ dK,$$

which is again 0, if $S > S_0$. It should now be clear that we may write the first equation as:

$$f(S) = [f(S_0) - f'(S_0)S_0] + f'(S_0)S + \int_{S_0}^{\infty} f''(K)(S - K)^+ dK + \int_0^{S_0} f''(K)(K - S)^+ dK$$

which is exactly the desired form.

1.e

Note that:

$$f(S) = (K_{strike} - A_T^a)^+ = (K_{strike} - g(T)(S_T)^a)^+.$$

I.e. the payoff is only non-zero if $S_T < \left(\frac{K}{g(T)}\right)^{\frac{1}{a}}$. Let $S_0 \rightarrow \infty$, then the Carr & Madan formula becomes a lot simpler:

$$f(S) = \int_0^{\infty} f''(K)(K - S)^+ dK$$

Note that for $K < \left(\frac{K_{strike}}{g(T)}\right)^{\frac{1}{a}}$, the function $f(K)$ and its derivatives can be determined as:

$$f(K) = K_{strike} - g(T)K^a$$

$$f'(K) = -ag(T)K^{a-1}$$

$$f''(K) = -a(a-1)g(T)K^{a-2}$$

Note furthermore that $f(K)$ has a kink at $\left(\frac{K_{strike}}{g(T)}\right)^{\frac{1}{a}}$, which shows up in the second derivative as a delta-function with the magnitude of the first derivative:

$$\begin{aligned} f(S) &= \int_0^{\infty} \left[a(1-a)g(T)K^{a-2}(K - S)^+ + ag(T)K^{a-1}\delta\left(K - \left(\frac{K_{strike}}{g(T)}\right)^{\frac{1}{a}}\right)(K - S)^+ \right] dK \\ &= \int_0^{\infty} a(1-a)g(T)K^{a-2}(K - S)^+ dK + ag(T)\left(\frac{K_{strike}}{g(T)}\right)^{\frac{a-1}{a}}\left(\left(\frac{K_{strike}}{g(T)}\right)^{\frac{1}{a}} - S\right)^+ \end{aligned}$$

The first term is only non-zero up until $\left(\frac{K_{strike}}{g(T)}\right)^{\frac{1}{a}}$, so we may simplify and discretize as:

$$\begin{aligned} f(S) &= \int_0^{\left(\frac{K_{strike}}{g(T)}\right)^{\frac{1}{a}}} a(1-a)g(T)K^{a-2}(K-S)^+ dK + ag(T)\left(\frac{K_{strike}}{g(T)}\right)^{\frac{a-1}{a}} \left(\left(\frac{K_{strike}}{g(T)}\right)^{\frac{1}{a}} - S\right)^+, \\ &= \sum_{i=1}^N a(1-a)g(T)K_i^{a-2}(K_i - S)^+ \Delta K + ag(T)\left(\frac{K_{strike}}{g(T)}\right)^{\frac{a-1}{a}} \left(\left(\frac{K_{strike}}{g(T)}\right)^{\frac{1}{a}} - S\right)^+, \end{aligned}$$

where $\Delta K = \frac{1}{N} \left(\frac{K_{strike}}{g(T)}\right)^{\frac{1}{a}}$. The first term represents a continuous portfolio of put options at different strikes, while the second term represent a single holding of put-options, with strike-price $\left(\frac{K_{strike}}{g(T)}\right)^{\frac{1}{a}}$. We can thus calculate the price from each term using standard Black-Scholes puts(see code), resulting in the following results for different discretization choices:

N	10	50	100	500	1000
Replication Price	0.2113	0.2150	0.2154	0.2157	0.2158

As we can see, the price converges to the same replication price we found in 1b as $N \rightarrow \infty$.

1.f

As seen in 1a, from the self-financing condition, the portfolio must satisfy:

$$dA_t^a = A_t^a[r dt + a\sqrt{V_t}dW_1^Q]$$

So A_t^a is simply a re-scaled heston-model. To see this explicitly let $U_t = a^2 V_t$ and expand with Itos:

$$\begin{aligned} dU_t &= a^2 dV_t = \kappa(a^2\theta - a^2 V_t)dt + a^2 \epsilon \sqrt{V_t} dW_2^Q \\ dU_t &= \kappa(a^2\theta - U_t)dt + a\epsilon \sqrt{U_t} dW_2^Q \end{aligned}$$

Thus A_t^a behaves exactly like the Heston-stock, simply with the substitution $\theta \rightarrow a^2\theta$ and $\epsilon \rightarrow a\epsilon$. Thus $e^{-rT}\mathbb{E}^Q[(K - A_T^a)^+]$ is simply the put price of a standard european put option in the Heston model. Using put call parity we find:

$$e^{-rT}\mathbb{E}^Q[(K - A_T^a)^+] = e^{-rT}\mathbb{E}^Q[(A_T^a - K)^+] - A_0^a + K e^{-rT}$$

By letting $A_0 = 1$ and $K = e^{rT}$ it is strikingly obvious that the put price is the same as the call price:

$$\mathbb{E}^Q[(K - A_T^a)^+] = \mathbb{E}^Q[(A_T^a - K)^+]$$

The put-price can now be calculated using the Heston call-price and it will be the arbitrage-free price of the insurance contract, because the portfolio A_t^a is self-financing, so $(K - A_t^a)^+$ is a contingent claim, whose price is given by the formula $e^{-rT}\mathbb{E}[\cdot]$.

1.g

I used the given Anderson and Lake python implementation [2] to determine the Heston call-price, which I found to be:

$$\mathbb{E}^Q[(A_t^a - e^{rT})^+] \approx 0.2039$$

You can see my code for details on this calculation.

1.h

I was unable to calculate the price of the static put option with the Heston pricing model, due to continuously getting the error 'negative partial sum did not converge' from the given Anderson and Lake implementation. However i would argue that we would not be able to replicate perfectly using this strategy. Consider if we were to once again write $A_t^a = g(t)(S_t)^a$. Start by calculating $d(S_t)^a$ using Ito:

$$\begin{aligned} d(S_t)^a &= a(S_t)^{a-1}dS_t + \frac{1}{2}a(a-1)(S_t)^{a-2}(dS_t)^2 \\ &= a(S_t)^{a-1}(rS_tdt + \sqrt{V_t}S_t dW_1^Q) + \frac{1}{2}a(a-1)(S_t)^{a-2}(V_t(S_t)^2dt) \\ &= ar(S_t)^a + a\sqrt{V_t}(S_t)^a dW_1^Q + \frac{1}{2}a(a-1)V_t(S_t)^a dt \\ &= a\left(r + \frac{1}{2}(a-1)V_t\right)(S_t)^a dt + a\sqrt{V_t}(S_t)^a dW_1^Q \end{aligned}$$

Now applying the product rule to dA_t^a yields

$$\begin{aligned} dA_t^a &= \frac{dg}{dt}(S_t)^a dt + g(t)d(S_t)^a \\ &= \frac{dg}{dt}(S_t)^a dt + g(t)\left(a\left(r + \frac{1}{2}(a-1)V_t\right)(S_t)^a dt + a\sqrt{V_t}(S_t)^a dW_1^Q\right) \end{aligned}$$

Once again using the self-financing condition:

$$dA_t^a = A_t^a[rdt + a\sqrt{V_t}dW_1^Q] = g(t)(S_t)^a[rdt + a\sqrt{V_t}dW_1^Q].$$

Simply matching the drift terms, we find:

$$\begin{aligned} \frac{dg}{dt}(S_t)^a + g(t)a\left(r + \frac{1}{2}(a-1)V_t\right)(S_t)^a &= g(t)(S_t)^a r \\ \frac{dg}{dt} &= \left(r - ar - \frac{1}{2}a(a-1)V_t\right)g(t) \end{aligned}$$

Thus we have $g(t) = e^{r(1-a)t - \frac{1}{2}a(a-1)\int_0^t V_{t'} dt'}$, and it is impossible to fully determine $g(t)$, because it depends on the stochastic variable V_t , making replication impossible. However if $a = 1$, then this stochastic term vanishes and it is once again possible to determine $g(t)$, which should make perfect replication feasible. I was however unable to test this with the given code.

1.i

In your notes[3] it is stated that for a call-option in the Heston model we have:

$$\Delta = P_1 = \frac{1}{2} + \frac{1}{\pi} \int_0^\infty \text{Re} \left[\frac{e^{i\phi \ln(K)} f_i(x, v, T, \phi)}{i\phi} d\phi \right]$$

Where f_i is the characteristic function given in the Heston-paper. As explained in page 4 of Gatherals book and Ch. 8 in Björk it is generally not possible to properly hedge if there are more 'sources of randomness' than traded assets. However i wasnt able to show this experimentally.

1.j

Define the value process as:

$$P_1 = h_S S_t + h_0 P_0 + h_B B_t$$

Where P represents a put option, S_t is a stock under the Heston model and B_t is the risk-free asset. Since S_t and B_t have 0 Vega, we can straightforwardly hedge the volatility:

$$\frac{\partial P_1}{\partial v} = h_0 \frac{\partial P_0}{\partial v}$$

$$h_0 = \frac{\partial P_1}{\partial v} \left(\frac{\partial P_0}{\partial v} \right)^{-1}$$

After hedging this risk, we can perform delta-hedging:

$$\frac{\partial P_1}{\partial s} = h_S \frac{\partial S}{\partial s} + h_0 \frac{\partial P_0}{\partial s}$$

$$\frac{\partial P_1}{\partial s} = h_S + \frac{\partial P_1}{\partial v} \left(\frac{\partial P_0}{\partial v} \right)^{-1} \frac{\partial P_0}{\partial s}$$

$$h_S = \frac{\partial P_1}{\partial s} - \frac{\partial P_1}{\partial v} \left(\frac{\partial P_0}{\partial v} \right)^{-1} \frac{\partial P_0}{\partial s}$$

Finally the remainder is to be invested into the risk-free asset:

$$h_B = \frac{P_1 - h_S S_t - h_0 P_0}{B_t}$$

Thus we can perfectly replicate the put with strike K_1 using this method in the continuous limit. At each timestep one must compute these derivatives for the current volatility- and stock-values, which is then used to calculate the h -values. Note that the order in which we calculate these 'fractional holdings' is crucial.

References

- [1] T. Björk, Arbitrage Theory in Continuous time, 4th edition. Oxford University Press (2020).
- [2] M. Frandsen and T. C. Pedersen, anderson-lake-python. Github (cloned 12-04-2026).
<https://github.com/tcpedersen/anderson-lake-python#>.
- [3] R. Poulsen, The Heston Formula: Notes, Quotes, and Codes.
https://www.dropbox.com/scl/fi/f3yd0w6j6t5a5ef4mi44y/Heston_Notes_Quotes_Codes.docx?rlkey=rkafzn0ytbho6f4b33eghd92l&e=1&dl=0

Hand-in 3

April 12, 2026

```
[15]: import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
```

1 1b

```
[125]: def Black_Scholes(S0, K, T, r, sigma, a):
    sigma = sigma * a
    d1 = (np.log(S0 / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    put_price = K * np.exp(-r * T) * stats.norm.cdf(-d2) - S0 * stats.norm.
↪cdf(-d1)
    return put_price

price = Black_Scholes(S0 = 1, K = np.exp(0.02 * 30), T = 30, r = 0.02, sigma = ↪
↪0.2, a = 0.5)
print(price)

def Hedge_simulator(n):
    m = 100 #Number of simulations
    T = 30
    dt = T / n
    results = np.zeros((m, 3))

    for i in range(m):
        #Initialize simulation
        S0 = 1
        A0 = 1
        K = np.exp(0.02 * 30)
        r = 0.02
        sigma = 0.2
        mu = 0.07
        a = 0.5

        Vpf = Black_Scholes(S0, K, T, r, sigma, a)
```

```

        d1 = (np.log(A0 / K) + (r + 0.5 * (sigma * a) ** 2) * T) / ((sigma * a) *
↪ np.sqrt(T))
        h_S = (stats.norm.cdf(d1) - 1) * a * A0 / S0
        h_B = Vpf - h_S * S0

        S = S0
        A = A0
        for j in range(n):
            dW = np.random.normal(0, np.sqrt(dt))
            S = S * np.exp((mu - 0.5 * sigma**2) * dt + sigma * dW)
            A = A * np.exp((a * mu + r - a * r - 0.5 * a**2 * sigma**2) * dt +
↪ a * sigma * dW)
            Vpf = h_S * S + h_B * np.exp(r * dt)

            #Rebalance the hedge
            tau = T - (j + 1) * dt # Time remaining
            if tau > 0:
                d1 = (np.log(A / K) + (r + 0.5 * (sigma * a) ** 2) * tau) /
↪ ((sigma * a) * np.sqrt(tau))
                h_S = (stats.norm.cdf(d1) - 1) * a * A / S
                h_B = Vpf - h_S * S

            #Save results
            results[i, 0] = Vpf
            results[i, 1] = max(K - A, 0)
            results[i, 2] = S

        return results

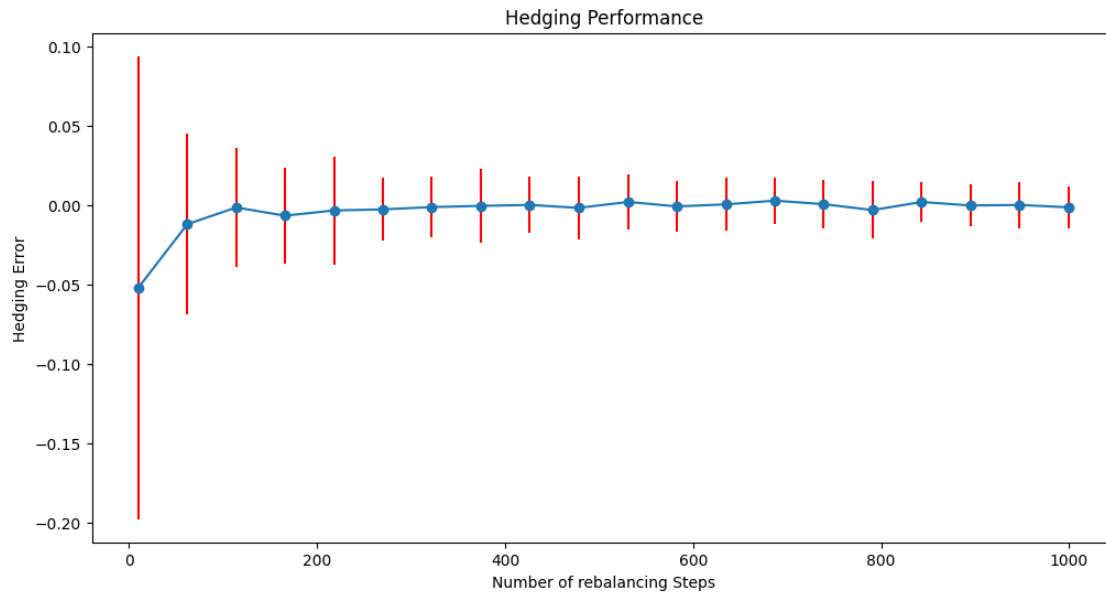
n_vals = np.linspace(10, 1000, 20, dtype=int)
errors = []
errors_std = []
for n in n_vals:
    results = Hedge_simulator(n)
    error = np.mean(results[:, 0] - results[:, 1])
    error_std = np.std(results[:, 0] - results[:, 1])
    errors.append(error)
    errors_std.append(error_std)

plt.figure(figsize=(12, 6))
plt.errorbar(n_vals, errors, yerr = errors_std, ecolor = 'red', marker='o')
#plt.plot(n_vals, errors, marker='o')
plt.xlabel('Number of rebalancing Steps')
plt.ylabel('Hedging Error')

```

```
plt.title('Hedging Performance')
plt.show()
```

0.215808770598381



2 1c

```
[17]: S0 = 1
      A0 = 1
      T = 30
      r = 0.02
      K = np.exp(r * T)
      sigma = 0.2
      mu = 0.07
      a = 0.5

      #First expression
      u = (np.log(K / A0) - (a * mu + r - a * r - 0.5 * a**2 * sigma**2) * T)/(sigma *
      ↪ a * np.sqrt(T))
      first = np.exp(-r * T) * K * stats.norm.cdf(u) - A0 * np.exp((a * mu - a * r) *
      ↪ T) * stats.norm.cdf(u - sigma * a * np.sqrt(T))
      print(first)

      #Second expression
      v = (np.log(K / S0**a) - (a * r - 0.5 * sigma**2 * a) * T)/(sigma * a * np.
      ↪ sqrt(T))
```

```

second = np.exp(-r * T) * K * stats.norm.cdf(v) - A0 * np.exp((a * r - r - 0.5 *
    ↪ sigma**2 * (a - a**2)) * T) * stats.norm.cdf(v - sigma * a * np.sqrt(T))
print(second)

#Third expression
d1 = (np.log(S0 / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * np.sqrt(T))
d2 = d1 - sigma * np.sqrt(T)
third = a * (K * np.exp(-r * T) * stats.norm.cdf(-d2) - S0 * stats.norm.
    ↪cdf(-d1))
print(third)

```

0.030442219432287962

0.4118609435922049

0.20805878961481744

3 1e

```

[114]: S0 = 1
        A0 = 1
        T = 30
        r = 0.02
        K = np.exp(r * T)
        sigma = 0.2
        a = 0.5

        gT = A0/(S0**a) * np.exp((r - a * r + 0.5 * a * (sigma**2) - 0.5 * a**2 *
            ↪(sigma**2)) * T)
        C = (K/gT)**(1/a)

        N_vals = [10, 50, 100, 500, 1000]
        delta_weight = a * gT * (C)**(a - 1)

        def f_double_prime(k):
            return a * (1 - a) * gT * k**(a - 2)

        for N in N_vals:
            strikes = np.linspace(0.0001, C, N)
            dk = strikes[1] - strikes[0]
            portfolio_price = sum(f_double_prime(k) * dk * Black_Scholes(S0, k, T, r,
            ↪sigma, a = 1) for k in strikes[:-1])
            portfolio_price += delta_weight * Black_Scholes(S0, C, T, r, sigma, a = 1)

            print(f"Replicated Portfolio Price: {portfolio_price} for N = {N}")

```

Replicated Portfolio Price: 0.2112653409383191 for N = 10

Replicated Portfolio Price: 0.21496727357130982 for N = 50

Replicated Portfolio Price: 0.21539200051616972 for N = 100

Replicated Portfolio Price: 0.21572604650269972 for N = 500
Replicated Portfolio Price: 0.215767447602271 for N = 1000

4 1g

```
[ ]: import sys
sys.path.append('anderson-lake-python-master')
from pricers import anderson_lake_expsinh
from models import HestonModel
from options import EuropeanCallOption

a = 0.5
r = 0.02
T = 30
V0 = a**2 * 0.2**2
theta = a**2 * 0.2**2
kappa = 2.0
epsilon = a * 1
rho = -0.5
A0 = 1
K = np.exp(r * T)

#Forward price
F = A0 * np.exp(r * T)

model = HestonModel(F, V0, kappa, theta, epsilon, rho, r)
option = EuropeanCallOption(T, K)

call = anderson_lake_expsinh(model, option)
print(f"Heston call price: {call}")
```

Heston call price: 0.20391690253223158

5 1h

```
[ ]: a = 1
K = np.exp(r * T)
gT = A0/(S0**a) * np.exp((r - a * r + 0.5 * a * (sigma**2) - 0.5 * a**2 *
    ↪(sigma**2)) * T)
C = (K / gT)**(1/a)

model_S = HestonModel(forward=S0*np.exp(r*T), vol=0.20, kappa=2.0,
                        theta=0.20, sigma=1.0, rho=-0.5, rate=r)

def get_put_price(k):
    opt = EuropeanCallOption(T, k)
```

```

    call = anderson_lake_expsinh(model_S, opt)
    return call

N = 100
strikes = np.linspace(0.1, C, N)
dk = strikes[1] - strikes[0]

integral_part = 0
for k in strikes[:-1]:
    integral_part += a * (1 - a) * gT * (k**(a - 2)) * get_put_price(k) * dk

delta_weight = a * gT * (C**(a - 1))
boundary_part = delta_weight * get_put_price(C)

replicated_price = integral_part + boundary_part
print(f"Replicated Portfolio Price: {replicated_price}")

```

```

-----
NoConvergenceError                                Traceback (most recent call last)
Cell In[119], line 23
    20     integral_part += a * (1 - a) * gT * (k**(a - 2)) * get_put_price(k)
    ↪ * dk
    22     delta_weight = a * gT * (C**(a - 1))
----> 23     boundary_part = delta_weight * get_put_price(C)
    25     replicated_price = integral_part + boundary_part
    26     print(f"Replicated Portfolio Price: {replicated_price}")

Cell In[119], line 11, in get_put_price(k)
     9     def get_put_price(k):
    10         opt = EuropeanCallOption(T, k)
----> 11         call = anderson_lake_expsinh(model_S, opt)
    12         return call

File c:\Users\Ben12\Desktop\Hand-In 3\anderson-lake-python-master\pricers.py:29
    ↪ in anderson_lake_expsinh(model, option)
    25     def anderson_lake_expsinh(model: HestonModel,
    26                                     option: EuropeanCallOption) -> float:
    27         scheme = ExpSinhQuadrature(init_step_size=0.5, error_tol=1e-12,
    28                                     max_iter=10000)
----> 29     return anderson_lake(model, option, scheme)

File c:\Users\Ben12\Desktop\Hand-In 3\anderson-lake-python-master\pricers.py:48
    ↪ in anderson_lake(model, option, scheme)
    45     dexp = np.exp(-x * tphi * omega + 1j * x * omega)
    46     return (dexp * Q(-1j * alpha + x * tphi) * tphi).real
----> 48     I = np.exp(alpha * omega) * scheme(integrand)
    49     R = model.forward * (alpha <= 0) - option.strike * (alpha <= -1) - \

```

```

50     0.5 * (model.forward * (alpha == 0) - option.strike * (alpha == -1)
51 return np.exp(- model.rate * option.tau) * (R - model.forward / np.pi *
↪I)

```

File c:\Users\Ben12\Desktop\Hand-In 3\anderson-lake-python-master\integration.p :

```

↪21, in ExpSinhQuadrature.__call__(self, func)
19 for n in range(self.max_iter):
20     old_estimate = new_estimate;
---> 21     new_estimate = self.compute_integral(func)
23     if abs(old_estimate - new_estimate) < self.error_tol:
24         break

```

File c:\Users\Ben12\Desktop\Hand-In 3\anderson-lake-python-master\integration.p :

```

↪72, in ExpSinhQuadrature.compute_integral(self, func)
70     threshold = abs(negative_n_sum * self.error_tol)
71 else:
---> 72     raise NoConvergenceError("negative partial sum did not converge")
74 # Zero n
75 zero_n_sum = self.eval_transformed_func(func, 0);

```

NoConvergenceError: negative partial sum did not converge